

PRACTICAL - 06

Name: Hansika Reddy Gunapati

Rollno: 2520030237

Section: 8

Part A: Client-Server Communication Using Named Pipes (FIFOs)

Aim

To create a client-server application using **Named Pipes (FIFOs)** where the client sends messages to the server, and the server processes and responds to the client.

Objective

- To understand IPC using Named Pipes.
- To implement client-server communication.
- To observe FIFO behavior with multiple clients.

Problem Statement

Create a client-server application using FIFOs. The client sends a message through a FIFO, the server reads and processes it, and sends a response back.

Theory

A **FIFO (First In, First Out)** is a named pipe used for communication between processes. Unlike an ordinary pipe, a FIFO has a name in the filesystem and can be used by unrelated processes.

Client → server_fifo → Server

Client ← response_fifo ← Server

When multiple clients use the same FIFO, messages are received according to the order in which their writes are completed. For reliable multiple-client communication, separate response FIFOs are preferable.

Algorithm

Server:

1. Create the FIFOs.
2. Open the server FIFO for reading.
3. Read the client message.
4. Process the message.
5. Send a response.
6. Repeat.

Client:

1. Open the server FIFO.
 2. Enter a message.
 3. Send the message.
 4. Read the server response.
 5. Display the response.
-

C Program – Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
```

```
#define SERVER_FIFO "server_fifo"
#define RESPONSE_FIFO "response_fifo"

int main()
{
    char buffer[100], response[150];
    int fd, response_fd;

    mkfifo(SERVER_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

    printf("Server started...\n");

    while (1)
    {
        fd = open(SERVER_FIFO, O_RDONLY);

        int bytes = read(fd, buffer, sizeof(buffer) - 1);

        if (bytes > 0)
        {
            buffer[bytes] = '\0';

            printf("Client Message: %s\n", buffer);
```

```

        snprintf(response, sizeof(response),
                 "Server processed: %s", buffer);

    close(fd);

    response_fd = open(RESPONSE_FIFO, O_WRONLY);

    write(response_fd, response, strlen(response) + 1);

    close(response_fd);

    printf("Response sent.\n\n");
}
}

return 0;
}

```

OUTPUT:

```

simra@simran315:~/fifo_experiment$ nano server.c
simra@simran315:~/fifo_experiment$ nano client.c
simra@simran315:~/fifo_experiment$ gcc server.c -o server
simra@simran315:~/fifo_experiment$ gcc client.c -o client
simra@simran315:~/fifo_experiment$ ls
client  client.c  server  server.c
simra@simran315:~/fifo_experiment$ ./server
Server started...
Waiting for client messages...
Client Message: Hello Server
Response sent.

```

C Program – Client

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <fcntl.h>

#include <unistd.h>


#define SERVER_FIFO "server_fifo"
#define RESPONSE_FIFO "response_fifo"


int main()
{
    char message[100], response[150];
    int fd, response_fd;


    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);


    message[strlen(message)] = '\0';


    fd = open(SERVER_FIFO, O_WRONLY);


    write(fd, message, strlen(message) + 1);
```

```

close(fd);

response_fd = open(RESPONSE_FIFO, O_RDONLY);

int bytes = read(response_fd, response, sizeof(response) - 1);

if (bytes > 0)
{
    response[bytes] = '\0';
    printf("Server Response: %s\n", response);
}

close(response_fd);

return 0;
}

```

OUTPUT:

```

simra@simran315:~$ cd fifo_experiment
simra@simran315:~/fifo_experiment$ ./client
Enter message: Hello Server
Server Response: Server processed: Hello Server
simra@simran315:~/fifo_experiment$ |

```

Observation

- Client successfully sent messages to the server.
- Server successfully processed and responded to messages.
- Multiple clients can use the same FIFO.

- The order of messages depends on the order of writes.
- A shared response FIFO can cause response-matching problems with multiple simultaneous clients.

Result

The client-server application was successfully implemented using **Named Pipes (FIFOs)**, and FIFO behavior with multiple clients was observed.

Part B: POSIX Signal Handling

Aim

To create a POSIX signal handling program that captures **SIGINT**, **SIGTERM**, and **SIGUSR1**, demonstrating asynchronous event handling.

Objective

- To understand POSIX signals.
- To implement signal handlers.
- To handle SIGINT, SIGTERM, and SIGUSR1.
- To understand asynchronous event handling.

Problem Statement

Create a C program that handles SIGINT, SIGTERM, and SIGUSR1 using POSIX signal handling.

Theory

A **signal** is an asynchronous notification sent to a process.

Signal	Meaning	How to generate
SIGINT	Interrupt	Ctrl + C
SIGTERM	Termination	kill -TERM PID

Signal	Meaning	How to generate
SIGUSR1	User-defined kill -USR1 PID	

A **signal handler** is a function that executes automatically when the corresponding signal is received.

Algorithm

1. Start the program.
 2. Register handlers for the three signals.
 3. Display the process PID.
 4. Wait for signals.
 5. Handle the received signal.
 6. Continue or terminate depending on the signal.
-

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int signal)
{
    if (signal == SIGINT)
    {
        printf("\nSIGINT received (Ctrl+C)\n");
    }
}
```

```

        printf("Program is still running.\n");
    }
    else if (signal == SIGUSR1)
    {
        printf("\nSIGUSR1 received.\n");
        printf("User-defined signal handled.\n");
    }
    else if (signal == SIGTERM)
    {
        printf("\nSIGTERM received.\n");
        printf("Terminating program...\n");
        exit(0);
    }
}

```

```

int main()
{
    struct sigaction action;

    action.sa_handler = signal_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    sigaction(SIGINT, &action, NULL);
    sigaction(SIGTERM, &action, NULL);
}

```

```

sigaction(SIGUSR1, &action, NULL);

printf("Signal handling program started.\n");
printf("Process PID: %d\n", getpid());

printf("Waiting for signals...\n");

while (1)
{
    pause();
}

return 0;
}

```

OUTPUT:

```

Server response: Server processed request: Server
simra@simran315:~/fifo_experiment$ nano signal_handler.c
simra@simran315:~/fifo_experiment$ gcc signal_handler.c -o signal_handler
simra@simran315:~/fifo_experiment$ ./signal_handler
Signal handling program started.
Process PID: 813
Waiting for signals...

SIGUSR1 received.
User-defined signal handled.
^C
SIGINT received (Ctrl+C)
Program is still running.

SIGTERM received.
Terminating program...
simra@simran315:~/fifo_experiment$ |

```

```
simra@simran315:~/fifo_experiment$ kill -USR1 813
simra@simran315:~/fifo_experiment$ kill -TERM 813
```

Observation

- SIGUSR1 was successfully handled asynchronously.
- SIGINT was captured when Ctrl+C was pressed.
- SIGTERM was captured and used to terminate the program.
- Signal handlers allow a process to respond to events without continuously checking for them.

Result

The POSIX signal handling program was successfully implemented and tested for **SIGINT**, **SIGTERM**, and **SIGUSR1**. The experiment demonstrated asynchronous event handling and the role of signal handlers.